0036. Valid Sudoku

Medium · Arrays & Hashing

leetcode.com/problems/valid-sudoku

Problem

- Decide whether a partially filled 9×9 Sudoku board is valid: no digit repeats in any row, column, or 3×3 box.
- Empty cells are '.'. The board need not be solvable.

Intuition

- Each filled cell belongs to exactly one row, one column and one box.
- Track seen digits per row, column and box; box index = (r/3)3 + c/3.
- A single scan finds any conflict.

Brute Force

- Check each row, each column and each box separately with a fresh set: 27 passes of 9 cells. Still O(81) but more code.

Optimal Approach

- Keep boolean[9][9] rows, cols, boxes (digit seen?).
- For each filled cell (r, c, d): b = (r/3)3 + c/3.
- If rows[r][d] or cols[c][d] or boxes[b][d] is already true, return false; else set all three.

Worked Example

```
Suppose the board has '8' at (0,0) and another '8' at (2,2) (classic invalid example).

(0,0) d=8 box=0 -> mark row0, col0, box0
...
(2,2) d=8 box=(2/3)*3+2/3 = 0 -> box0 already has 8 -> return false
```

Java Solution

```java
public class ValidSudoku {
    public boolean isValidSudoku(char[][] board) {
        boolean[][] rows = new boolean[9][9], cols = new boolean[9][9], boxes = new boolean[9][9];
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                char ch = board[r][c];
                if (ch == '.') continue;
                int d = ch - '1', b = (r / 3) * 3 + c / 3;
                if (rows[r][d] || cols[c][d] || boxes[b][d]) return false;
                rows[r][d] = cols[c][d] = boxes[b][d] = true;
            }
        }
        return true;
    }
}
```

Complexity

- Time: O(81) = O(1) for a fixed board; O(n^2) for an n x n generalisation.
- Space: O(1) (three 9×9 tables).

Pitfalls

- Getting the box formula wrong; test it on corners (0,0)→0, (8,8)→8, (4,7)→5.
- Do not try to solve the puzzle, only validate.